

Agents

Language Models

Course Contents

Deep Learning

Transformer

LLMs

Context Engineering

Finetuning

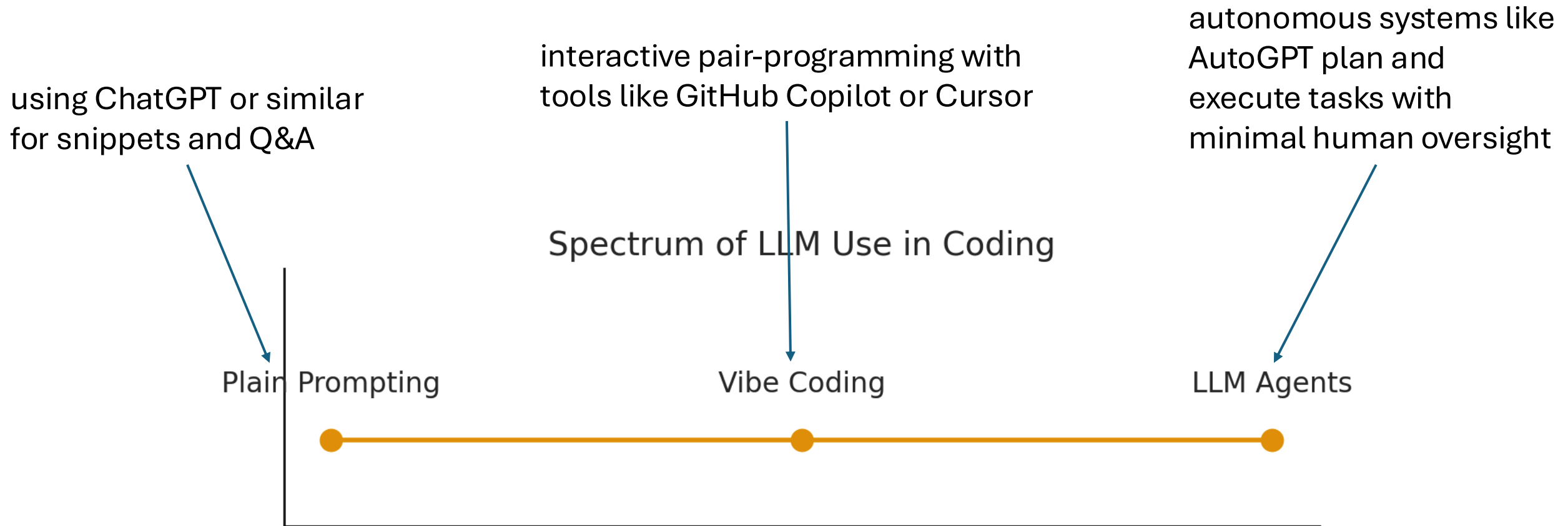
Agents

The Bigger Picture

This Part: Agents

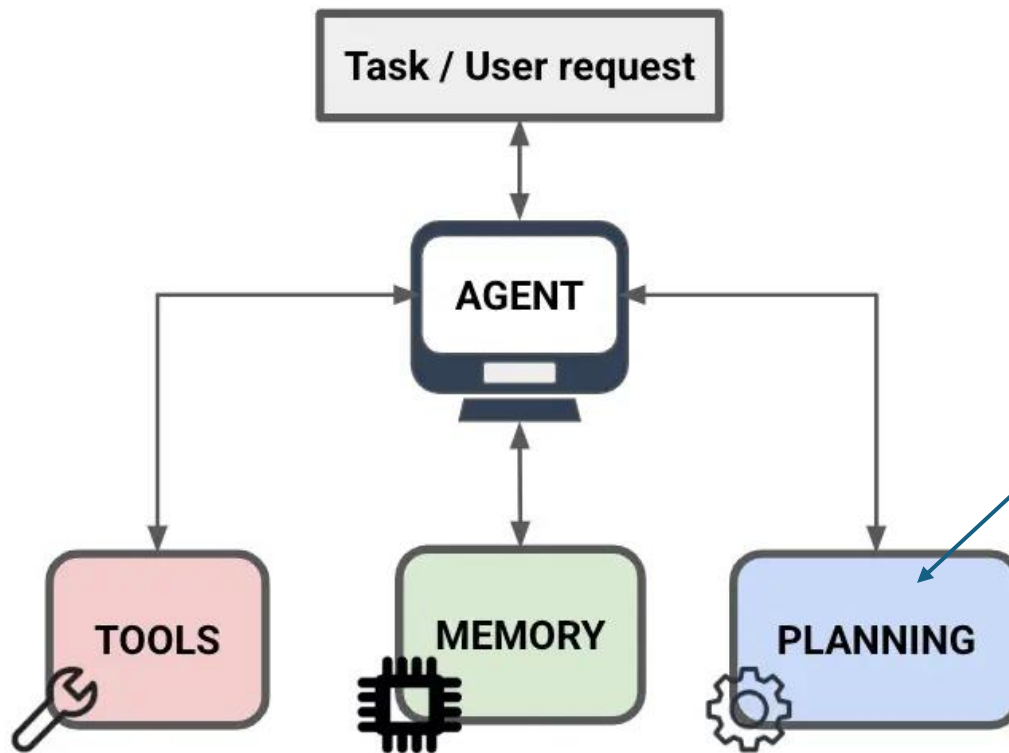
- tool use
- memory
- reasoning
- LLM orchestration
- planning
- multi-agent systems

Example: LLM Use in Coding



Agentic Intelligence

goal: autonomous problem-solving



decomposition of complex task in simpler steps
(uses reasoning capabilities)

Intuitive and Analytical Thinking

two systems of human thinking (from Kahneman, *Thinking, Fast and Slow*):

- system 1: fast, automatic, intuitive, effortless, often subconscious
- system 2: slow, deliberate, analytical, effortful, conscious

LLMs analogous to system 1 (although without true intuition or understanding), lacking the depth of reflective, analytic reasoning

→ for agentic intelligence, need to enhance LLMs with system 2 capabilities, including tool usage and fact-checking

ReAct

prompting technique instructing an LLM to interleave reasoning steps (CoT-like) with actions (tool use):

- reasoning (Re): model reflects, analyzes the problem, breaks it down, and plans
- acting (Act): model performs an external action (e.g., querying a database, calling an API, using a calculator, searching the web)
- process loops: reasoning → action → observation → updated reasoning → next action → ... until the task is completed

ReAct Prompt Template

zero-shot ReAct:

You are an intelligent assistant that can reason and take actions.
For each step, follow this format:

Reasoning: [your thought process here]

Action: [the action you will take, if any]

Observation: [the result of the action]

Repeat as needed until you can confidently provide the answer.

At the end, give your final answer in this format:

Final Answer: [your final response here]

Task: [insert the problem/question here]

often few-shot ReAct:
include 1–3 worked-out
examples of ReAct loop

→ improves reliability on
more complex, multi-step
tasks.

ReAct Example in Use

⋮

Task: What is the square root of the population of Tokyo (approximately)?

response of the model:

Reasoning: I need to find the population of Tokyo first, then compute its square root.

Action: Search "Tokyo population 2023"

Observation: Tokyo's population is about 37.4 million.

Reasoning: Now I calculate the square root of 37.4 million.

Action: Calculator: $\text{sqrt}(37,400,000)$

Observation: Result is about 6116.

Final Answer: The square root of Tokyo's population (~37.4 million) is approximately 6116.

Tool Usage

crucial LLM capabilities:

1. know how to access different tools
 - code interpreters
 - web search
 - calculators (to overcome lack of mathematical understanding)
 - ...
2. select right tool at right time

example: OpenAI

Available tools

Here's an overview of the tools available in the OpenAI platform—select one of them for further guidance on usage.

-  **Function calling**
Call custom code to give the model access to additional data and capabilities.
-  **Web search**
Include data from the Internet in model response generation.
-  **Remote MCP servers**
Give the model access to new capabilities via Model Context Protocol (MCP) servers.
-  **File search**
Search the contents of uploaded files for context when generating a response.
-  **Image generation**
Generate or edit images using GPT Image.
-  **Code interpreter**
Allow the model to execute code in a secure container.
-  **Computer use**
Create agentic workflows that enable a model to control a computer interface.

Acquistion of Tool Usage Skills

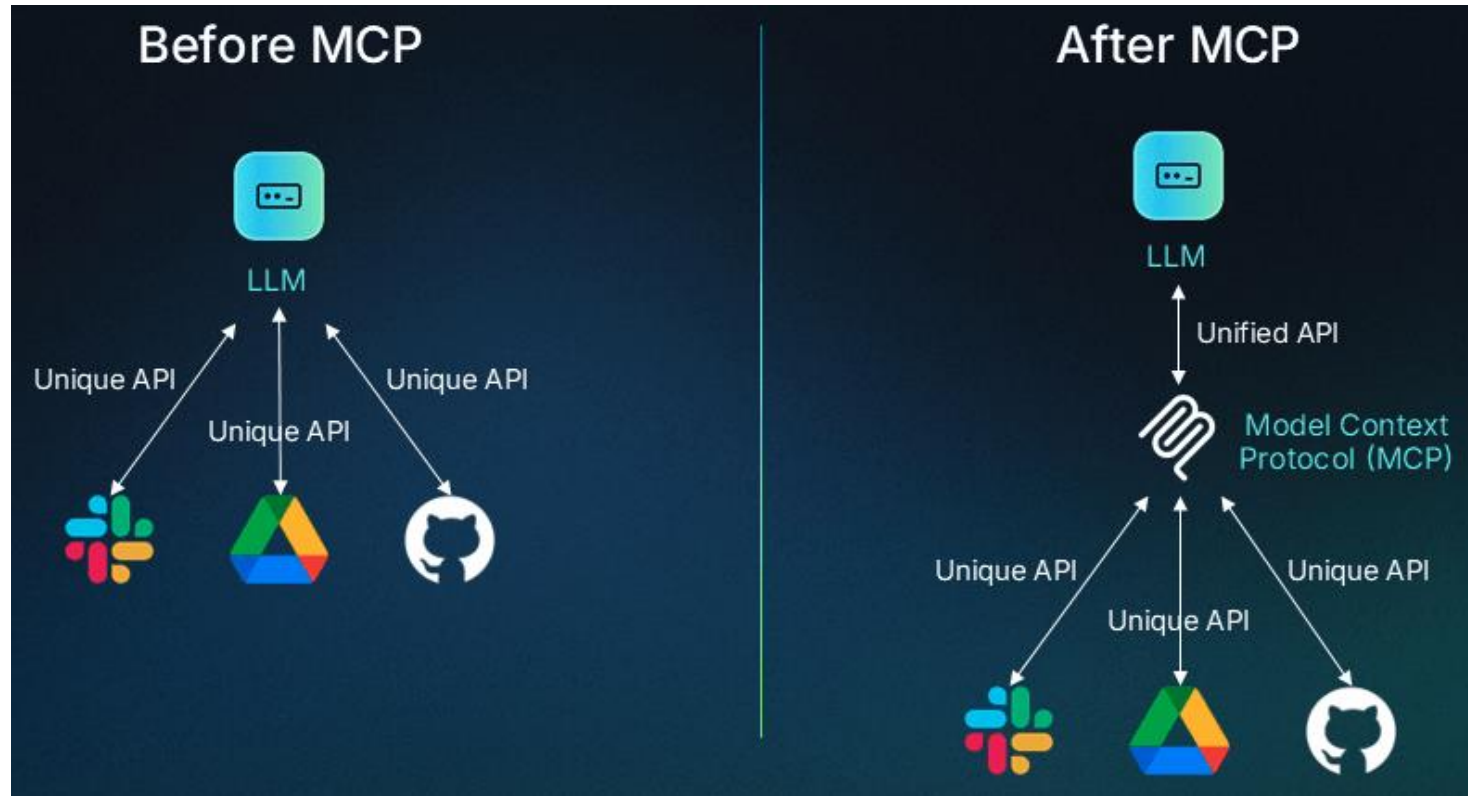
using instruction-tuned LLMs (like ChatGPT) as agent:

- core capability (*how* to use a tool) comes from pretraining and different finetuning stages
- tool access controlled at runtime via prompting and system messages
- tool selection (*when* to use *which* tool) decided from context

potentially, custom finetuning for specific tools or workflows

Model Context Protocol (MCP)

open protocol to connect LLMs to different tools or data sources
standardization for provisioning of external context



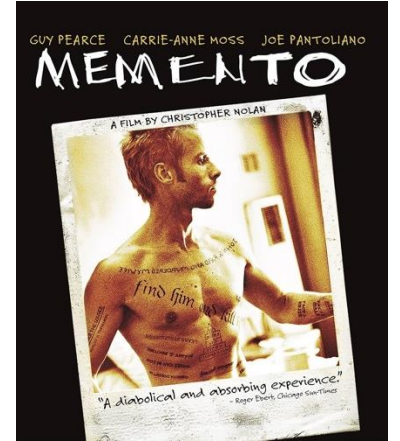
Test-Time Scaling

improve a model's performance (especially reasoning capabilities) without retraining it, by allocating more compute during inference time:

- more complex prompting strategies (CoT, self-consistency, prompt optimization, ...)
- extensive tool use

→ test-time compute (number of samples, reasoning steps) as another scaling law (in addition to parameters, training data, training compute)

LLM/Agent Memory



short-term memory: context

- chatbot conversation: typically just append each turn
- for conversations longer than context window, older turns need to be summarized or truncated
- forgotten after each conversation

long-term memory: survives across conversations

- typically implemented through external vector store and context injection
→ RAG mechanism
- expensive alternative: per-instance/per-agent adaption ([test-time training](#))
→ only update (i.e., finetune) small adapters (e.g., LoRA)

Reasoning Models

some reasoning capabilities:

standard LLMs using prompting strategies such as CoT or ReAct

→ reasoning emerges implicitly from pretraining (performance can be brittle and highly prompt-dependent)

better reasoning capabilities:

reasoning-specific LLMs explicitly trained or finetuned on reasoning objectives (often with CoT-style supervision or multi-step reasoning datasets)

→ learn to reason by default, not just when prompted

examples: [OpenAI o1](#), [DeepSeek-R1](#) (open source)

LLM Orchestration and Agents

wrappers around LLM calls to make them more useful in real-world applications

LLM orchestration:

pipelines + prompt templates + access to tools & memory + context management
→ structured environment

e.g., LangChain, LlamaIndex

agent systems:

LLM orchestration + autonomy/planning loop

→ kind of autopilot on top of orchestration layer

e.g., AutoGPT, OpenAgents, BabyAGI

Planning

scaffolding around (reasoning-strong) LLM, structuring acting in sequence:

- break goals into subgoals
- use the LLM to propose or refine steps
- optionally search through multiple alternatives
- execute actions and adapt plans dynamically

implementation options:

- use LLM itself as a planner via prompting (CoT, ReAct)
- search-based planning
- control loop (LLM as policy generator) → AutoGPT, etc

Multi-Agent Systems

idea:

complex, distributed, and dynamic problems can be solved more efficiently, flexibly, and robustly by using multiple specialized agents rather than a single one

→ extended agent system:

LLM orchestration + autonomy/planning loop

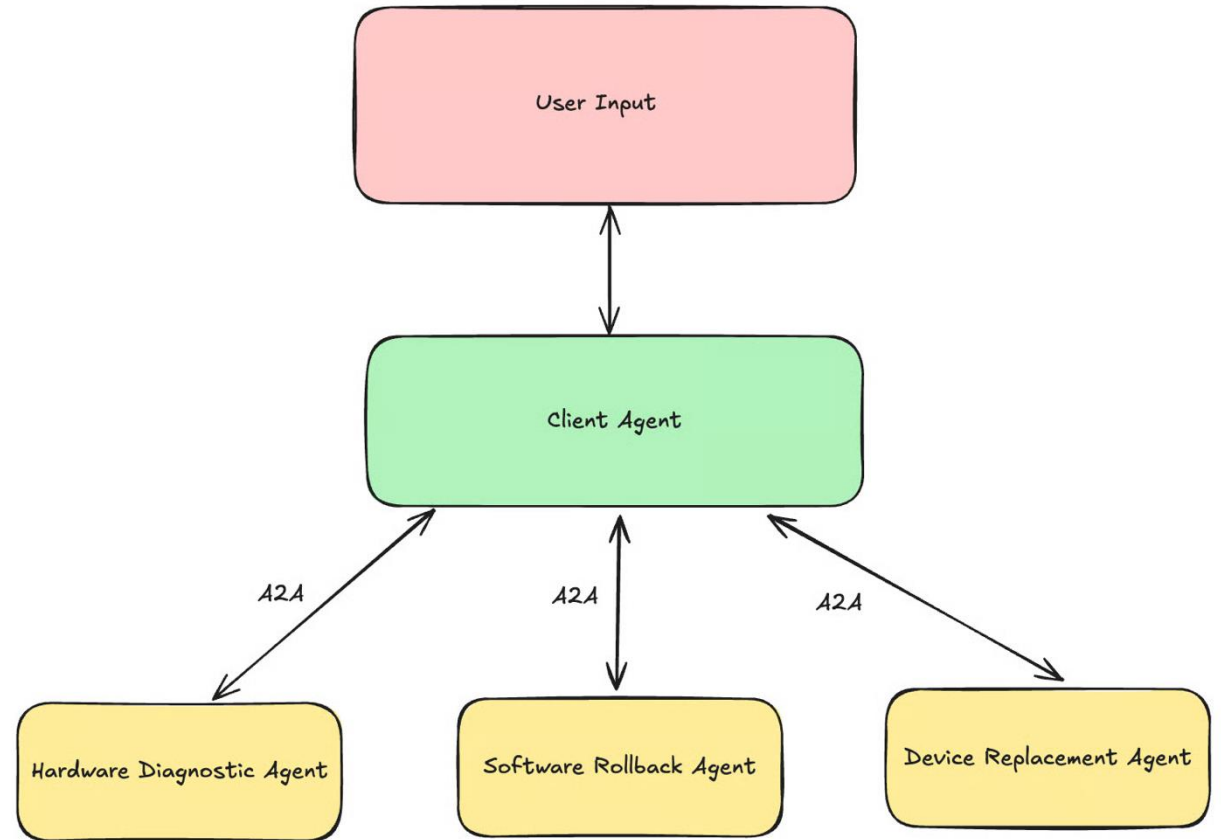
+ coordination between multiple agents

e.g., AutoGen

Agent2Agent Protocol (A2A)

open standard for enabling communication, collaboration, and interoperability between autonomous AI agents (complementary to MCP)

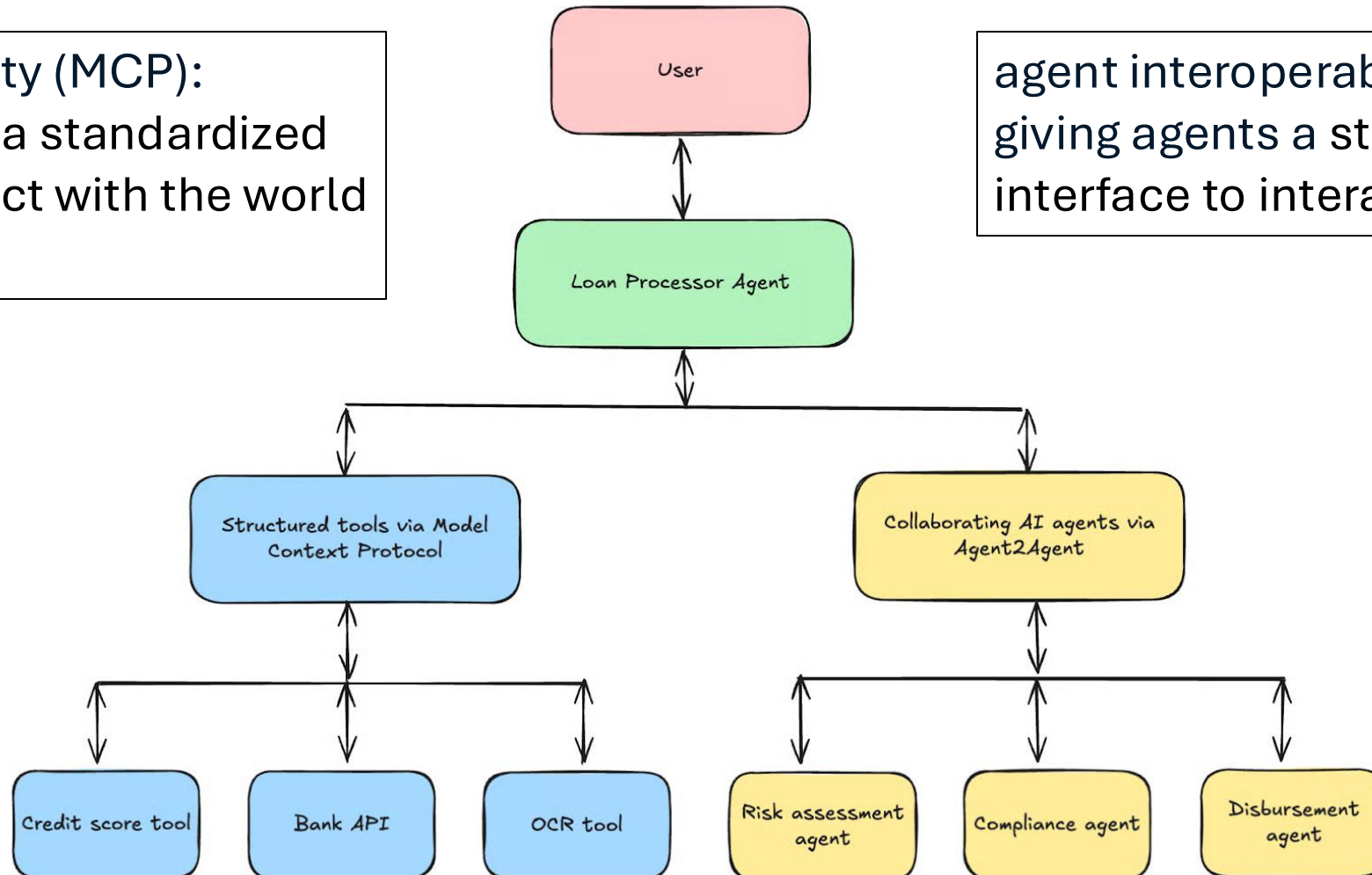
example:



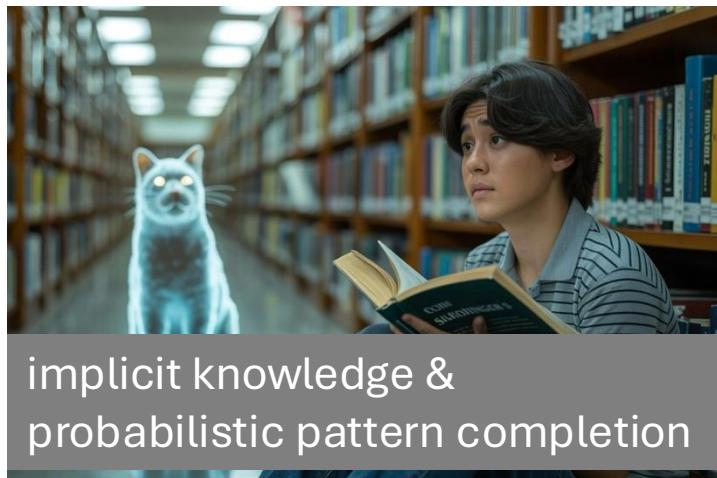
Example for Combination of A2A and MCP

tool interoperability (MCP):
giving each agent a standardized
interface to interact with the world
(tools, data, APIs)

agent interoperability (A2A):
giving agents a standardized
interface to interact with each other

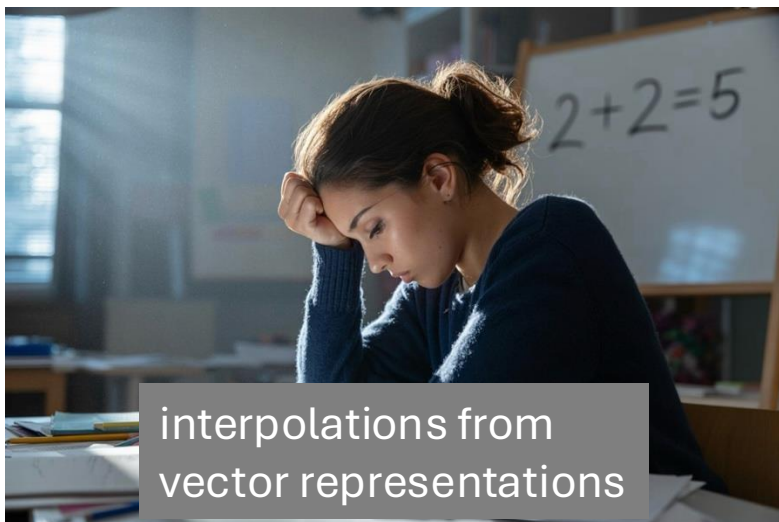


hallucinations



LLM: *kind of a lossy simulation
of a savant with cognitive issues*

encyclopedic knowledge



jagged intelligence



anterograde amnesia

Exercise: Tool Use & Planning

Build a simple LLM agent:

- choose an agentic task (math, data analysis, file organization, trip planning, ...)
- define the agent & planning strategy (prompt templates, tool call and planning instructions)
- provide tools the agent can call (functions, APIs, calculator, ...)
- implement the reasoning/planning loop (ReAct, reflexion, ...)
- optionally add short-term memory (scratchpad or state tracking)
- evaluate robustness and correctness (planning and reasoning quality, task success rate, ...)